

Programación Lineal I

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

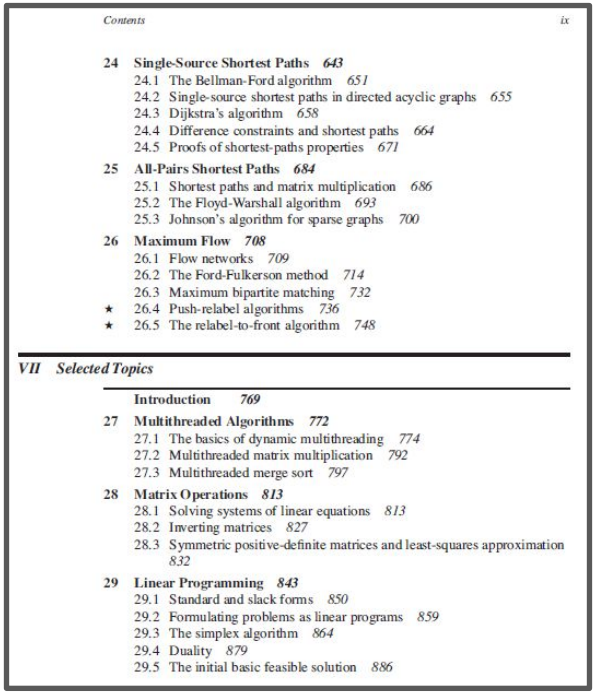
¿Qué es la Programación Lineal?



Programación Lineal: Introducción

- Es una técnica para resolver problemas de optimización (maximización / minimización)
- Origen en la II Guerra Mundial (¿Qué es lo mejor que podemos hacer con los recursos que tenemos?)
- Es parte central de la disciplina Investigación Operativa
- Es un planteo matemático, no algorítmico.

Programación Lineal: Bibliografía



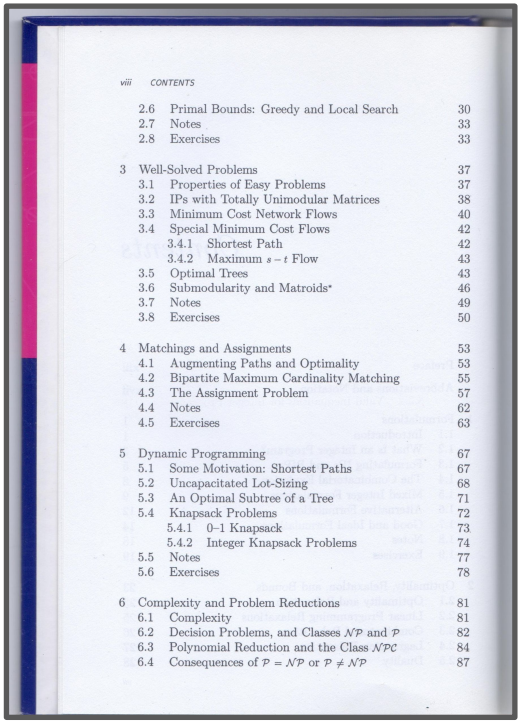
Contents ix

24 Single-Source Shortest Paths 643
24.1 The Bellman-Ford algorithm 651
24.2 Single-source shortest paths in directed acyclic graphs 655
24.3 Dijkstra's algorithm 658
24.4 Difference constraints and shortest paths 664
24.5 Proofs of shortest-paths properties 671
25 All-Pairs Shortest Paths 684
25.1 Shortest paths and matrix multiplication 686
25.2 The Floyd-Warshall algorithm 693
25.3 Johnson's algorithm for sparse graphs 700
26 Maximum Flow 708
26.1 Flow networks 709
26.2 The Ford-Fulkerson method 714
26.3 Maximum bipartite matching 732
★ 26.4 Push-relabel algorithms 736
★ 26.5 The relabel-to-front algorithm 748

VII Selected Topics

Introduction 769
27 Multithreaded Algorithms 772
27.1 The basics of dynamic multithreading 774
27.2 Multithreaded matrix multiplication 792
27.3 Multithreaded merge sort 797
28 Matrix Operations 813
28.1 Solving systems of linear equations 813
28.2 Inverting matrices 827
28.3 Symmetric positive-definite matrices and least-squares approximation 832
29 Linear Programming 843
29.1 Standard and slack forms 850
29.2 Formulating problems as linear programs 859
29.3 The simplex algorithm 864
29.4 Duality 879
29.5 The initial basic feasible solution 886

Introduction to Algorithms, Cormen et al.



viii CONTENTS

2.6 Primal Bounds: Greedy and Local Search 30
2.7 Notes 33
2.8 Exercises 33

3 Well-Solved Problems 37
3.1 Properties of Easy Problems 37
3.2 IPs with Totally Unimodular Matrices 38
3.3 Minimum Cost Network Flows 40
3.4 Special Minimum Cost Flows 42
3.4.1 Shortest Path 42
3.4.2 Maximum $s-t$ Flow 43
3.5 Optimal Trees 43
3.6 Submodularity and Matroids* 46
3.7 Notes 49
3.8 Exercises 50

4 Matchings and Assignments 53
4.1 Augmenting Paths and Optimality 53
4.2 Bipartite Maximum Cardinality Matching 55
4.3 The Assignment Problem 57
4.4 Notes 62
4.5 Exercises 63

5 Dynamic Programming 67
5.1 Some Motivation: Shortest Paths 67
5.2 Uncapacitated Lot-Sizing 68
5.3 An Optimal Subtree of a Tree 71
5.4 Knapsack Problems 72
5.4.1 0–1 Knapsack 73
5.4.2 Integer Knapsack Problems 74
5.5 Notes 77
5.6 Exercises 78

6 Complexity and Problem Reductions 81
6.1 Complexity 81
6.2 Decision Problems, and Classes $\mathcal{NP}$ and $\mathcal{P}$ 82
6.3 Polynomial Reduction and the Class $\mathcal{NPC}$ 84
6.4 Consequences of $\mathcal{P} = \mathcal{NP}$ or $\mathcal{P} \neq \mathcal{NP}$ 87

Integer Programming, L. Wolsey

Programación Lineal: Primer problema



Programación Lineal: Primer problema

- La heladería “Fa Caldo” fabrica dos tipos de helado: de agua y de crema.
- Cada lata de helado de agua lleva 2 kilos de azúcar y 2 kilos de almidón de maíz.
- Cada lata de helado de crema lleva 2 kilos de azúcar,4 kilos de crema y 4 kilos de almidón de maíz.
- Se dispone mensualmente de 600 kilos de azúcar,600 kilos de crema y 800 kilos de almidón.
- El beneficio por lata es de 8 pesos para los helados de agua y de 10 para los de crema.

Programación Lineal: Primer problema

Variables:

$X1$: Latas de helado de agua producidas y vendidas por mes

$X2$: Latas de helado de crema producidas y vendidas por mes

Restricciones:

$2\,X1 + 2\,X2 <= 600$ [Kg azúcar / mes]

$4\,X2 <= 600$ [Kg crema / mes]

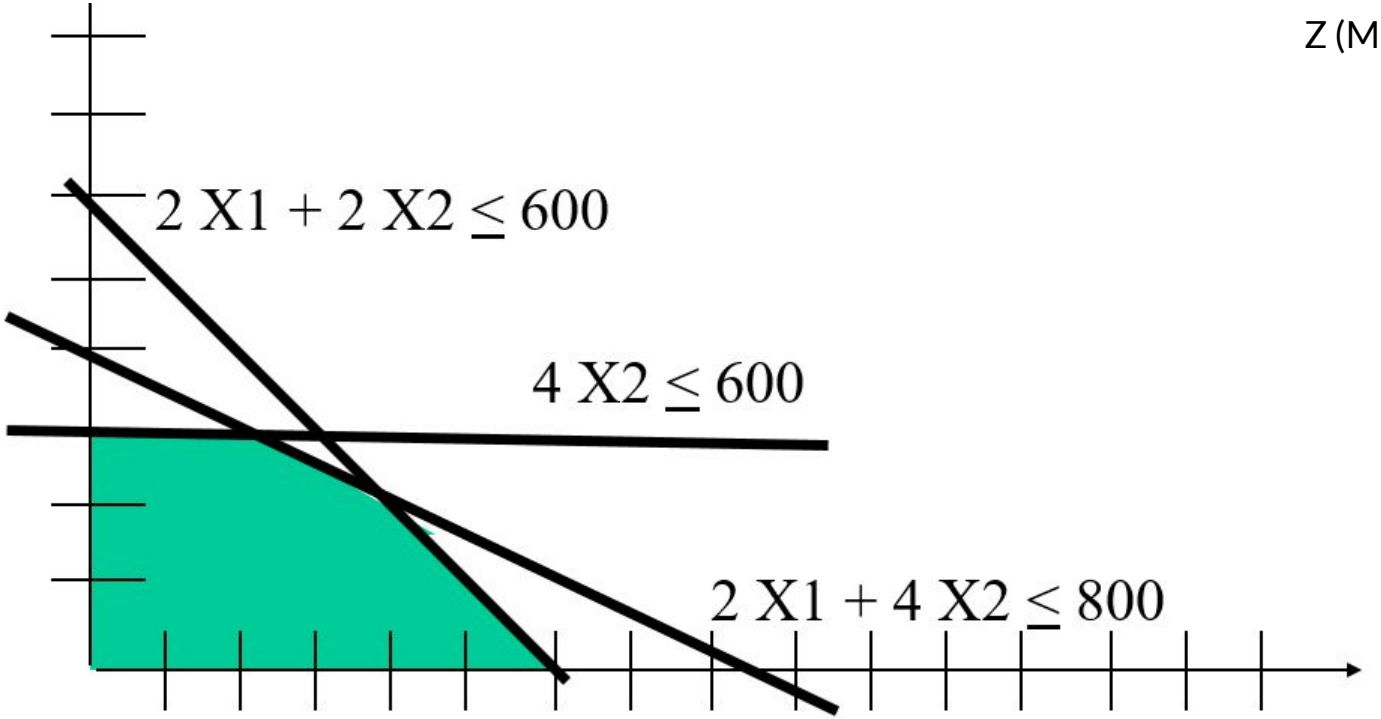
$2\,X1 + 4\,X2 <= 800$ [Kg almidón/mes]

Objetivo:

$Z\,(Max) = 8\,X1 + 10\,X2$

Programación Lineal: Primer problema

Z (Max) = 8 X1 + 10 X2



Programación Lineal: Condiciones Fundamentales



George Dantzig (1914-2005)
Créditos:
https://mathshistory.st-andrews.ac.uk/Biographies/Dantzig_George/

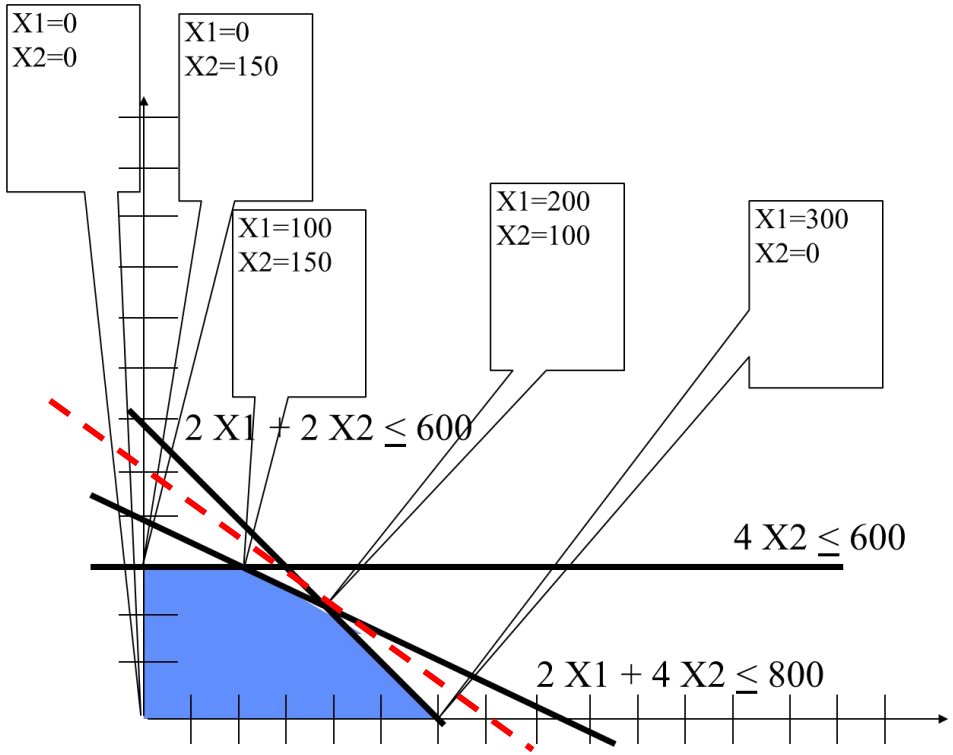
Programación Lineal: Condiciones Fundamentales

- Todas las restricciones son lineales
- La función objetivo es lineal
- Todas las variables son positivas y continuas
- Todas las restricciones incluyen la igualdad



Si cumplimos estas condiciones, la solución óptima (si existe) estará en un vértice

Programación Lineal: Primer problema



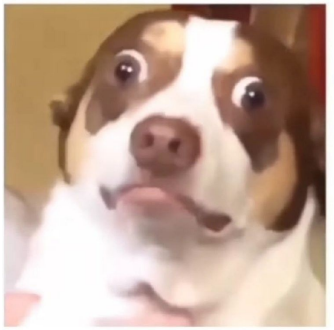
Z (Max) = 8 X1 + 10 X2

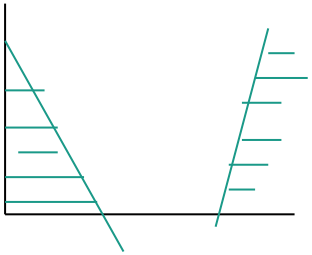
Programación Lineal: Finales posibles



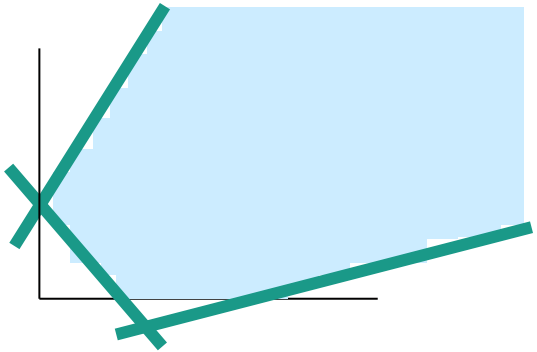
Si cumplimos estas condiciones, la solución óptima (si existe) estará en un vértice

¿Si existeeee?

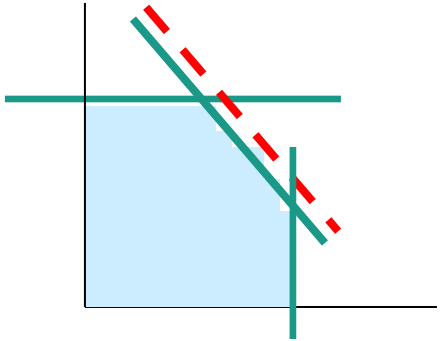




Incompatible



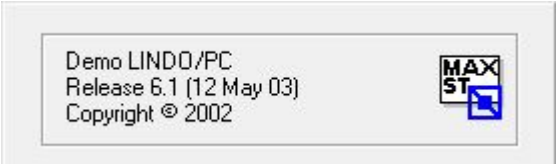
Poliedro Abierto

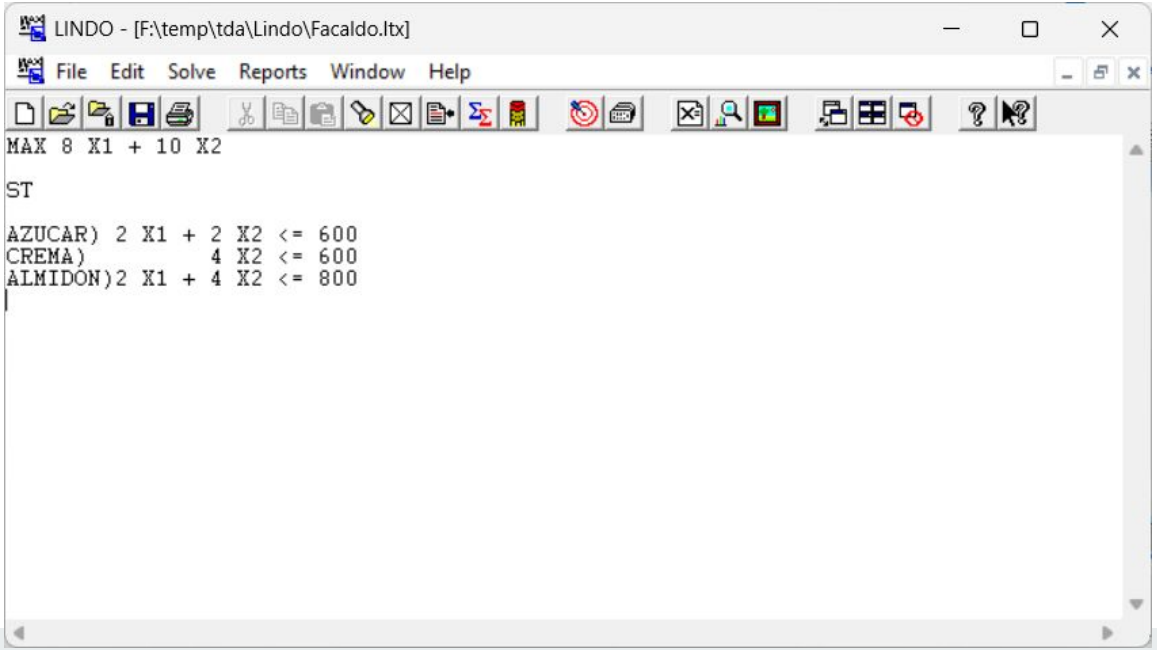


Soluciones Alternativas

Programación Lineal: Software de resolución

- Vamos a ver la resolución en el software LINDO
- www.lindo.com (Demo Version)





Programación Lineal: Software de resolución

LINDO - [Reports Window]

File Edit Solve Reports Window Help

```
LP OPTIMUM FOUND AT STEP      0

        OBJECTIVE FUNCTION VALUE

        1)      2600.000

 VARIABLE        VALUE          REDUCED COST
       X1      200.000000          0.000000
       X2      100.000000          0.000000


      ROW    SLACK OR SURPLUS     DUAL PRICES
   AZUCAR)         0.000000          3.000000
    CREMA)       200.000000          0.000000
 ALMIDON)          0.000000          1.000000

NO. ITERATIONS=       0

RANGES IN WHICH THE BASIS IS UNCHANGED:
```

Demo LINDO/PC
Release 6.1 (12 May 03)
Copyright © 2002

LINDO - [Reports Window]

File Edit Solve Reports Window Help

```
 ALMIDON)          0.000000          1.000000

NO. ITERATIONS=       0

RANGES IN WHICH THE BASIS IS UNCHANGED:

                        OBJ COEFFICIENT RANGES
VARIABLE          CURRENT        ALLOWABLE        ALLOWABLE
                   COEF          INCREASE         DECREASE
      X1         8.000000         2.000000         3.000000
      X2        10.000000         6.000000         2.000000

                        RIGHTHAND SIDE RANGES
     ROW          CURRENT        ALLOWABLE        ALLOWABLE
                    RHS          INCREASE         DECREASE
  AZUCAR       600.000000       200.000000       100.000000
   CREMA       600.000000         INFINITY       200.000000
 ALMIDON       800.000000       100.000000       200.000000
```

Programación Lineal: Software de resolución

- Otros softwares:
  - Glpk (open source)
  - Cplex (propietario de IBM, licencia estudiantil)
  - Concorde (Específico para TSP)
  - Hay bibliotecas para Python, C, etc…
    - Vamos a usar la biblioteca PuIP: https://coin-or.github.io/pulp/index.html

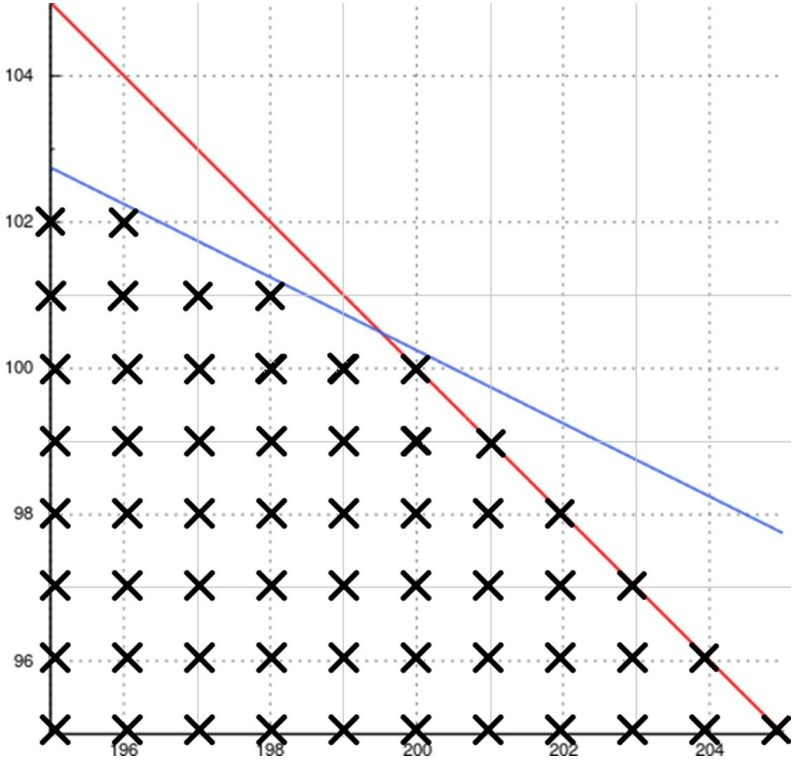
Programación Lineal Entera



Programación Lineal Entera

- Define variables que sólo pueden tomar valor entero:
  - No puedo fabricar medio barco
  - Redondear para fabricar uno o ninguno puede estar muy lejos del óptimo (o ser incompatible)
- También define variables con valor 0-1 (bivalentes)
  - Útiles como variables indicativas: si se hace o no una actividad
  - También como variables de decisión: ¿Lo hago o no lo hago?
- Parece una buena idea, pero…..

Programación Lineal Entera



- El óptimo no necesariamente está en un vértice
- Simplex no me alcanza, debo buscar otros métodos de resolución
  - Planos de corte
  - Branch and bound
- Mucho más complejos

Programación Lineal Entera: Variables de decisión

Tenemos que decidir si lanzamos diferentes productos (tenemos 6). Se deben lanzar al menos 3. Cada producto tiene un determinado costo de lanzamiento (y tenemos un presupuesto):

Yi = Variable que indica si se lanza el producto i:
Vale 1 si se lanza
Vale 0 si no se lanza

$$\sum_i y_i \geq 3$$

$$Z_{min} = \sum c_i y_i$$

Programación Lineal Entera: Variables de decisión

Si lanzamos el producto 5 o 6, debemos lanzar el 1 sí o sí. ¿Cómo lo representamos?

$$y_1 \geq y_5$$
$$y_1 \geq y_6$$

Si se lanzan juntos los productos 3 y 4, se ahorran $100. ¿Cómo lo representamos?

$$y_{ahorro} \leq y_3$$
$$y_{ahorro} \leq y_4$$

equivalente a...

$$2 \cdot y_{ahorro} \leq y_3 + y_4$$

Y en el funcional, agregamos: Zmin = ... - $100 Yahorro

Programación Lineal Entera: Variables de decisión

- Estructuras habituales

OR de n variables: $$Y_{or} \leq \sum_{i=1}^{n} y_i \leq n \cdot Y_{or}$$

AND de n variables: $$n \cdot Y_{and} \leq \sum_{i=1}^{n} y_i \leq (n-1) + Y_{and}$$

(En ambos casos n es una constante para el problema)

Programación Lineal Entera:

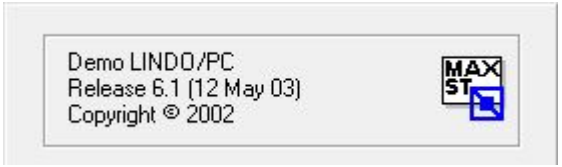
- Problema de la mochila

Tenemos un conjunto de elementos a guardar en una mochila, cada uno con un valor v_i y peso w_i, no podemos excedernos en peso W. Obtener el máximo valor que podemos guardar en la mochila. (nota: w_i y v_i son constantes, no variables)

Y_i variables booleanas.

$$\sum w_i Y_i <= W$$

$$MAX \sum v_i Y_i$$





Programación Lineal Entera:

LINDO - [F:\temp\tda\Lindo\mochila.ltx]

File Edit Solve Reports Window Help

```
MAX 30 Y1 + 15 Y2 + 80 Y3 + 100 Y4 + 10 Y5

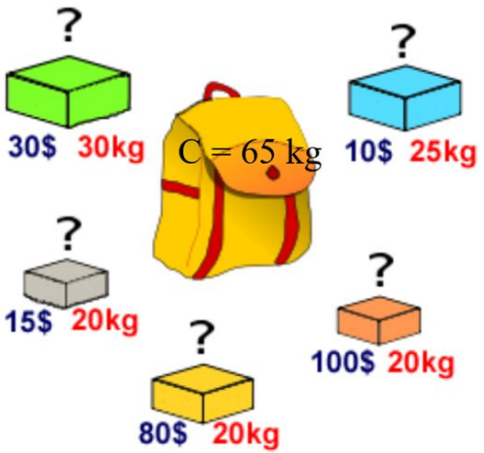
ST

CAP) 30 Y1 + 20 Y2 + 20 Y3 + 20 Y4 + 25 Y5 <= 65

END

INT 5
```

Demo LINDO/PC
Release 6.1 (12 May 03)
Copyright © 2002



Programación Lineal Entera:

LINDO - [Reports Window]

File Edit Solve Reports Window Help

```
LP OPTIMUM FOUND AT STEP      6
OBJECTIVE VALUE =    205.000000


NEW INTEGER SOLUTION OF    195.000000     AT BRANCH       0 PIVOT       6
RE-INSTALLING BEST SOLUTION...

       OBJECTIVE FUNCTION VALUE

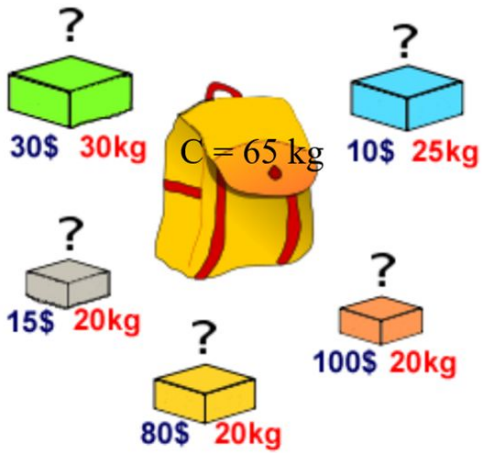
       1)       195.0000

 VARIABLE         VALUE          REDUCED COST
       Y1          0.000000        -30.000000
       Y2          1.000000        -15.000000
       Y3          1.000000        -80.000000
       Y4          1.000000       -100.000000
       Y5          0.000000        -10.000000


      ROW    SLACK OR SURPLUS      DUAL PRICES
      CAP)          5.000000          0.000000

NO. ITERATIONS=       6
BRANCHES=     0 DETERM.=  1.000E     0
```

Demo LINDO/PC
Release 6.1 (12 May 03)
Copyright © 2002



Resolución de Programación Lineal Entera: Problema del Viajante de Comercio



Programación Lineal Entera: Viajante

- *¿Cuáles son las variables?*
- *¿Cuál es el objetivo?*
- *¿Cuáles son las restricciones?*

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Programación Lineal Entera: Viajante

- *Determinar en qué orden visitar las ciudades.*
- *Para minimizar el costo del viaje.*
- *Visitando cada una exactamente una vez y volviendo al inicio.*

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

Programación Lineal Entera: Viajante

- Y_{ij}: *Vale 1 si hago el trayecto de i a j* $\quad \forall\ i,j: 0...N$
- $Z(Min) = \sum_i \sum_j C_{ij} \cdot Y_{ij}$ $\quad \forall\ i,j: 0...N$
  (C_{ij} *es constante*)
- *¿Cómo lo obligo a recorrer todas?*

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

N es la cantidad de ciudades a visitar, además del origen. En este ejemplo, es 4

Programación Lineal Entera: Viajante

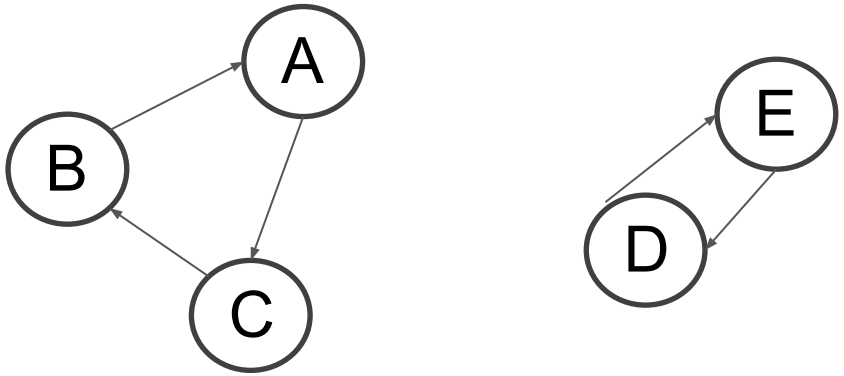
- *De cada ciudad salgo una vez:*
- $\sum_j Y_{ij} = 1 \qquad \forall\, i: 0 \ldots N$
- *A cada ciudad llego una vez:*
- $\sum_i Y_{ij} = 1 \qquad \forall\, j: 0 \ldots N$

	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

¿Es suficiente?

Programación Lineal Entera: Viajante

- *Una solución podría ser:*

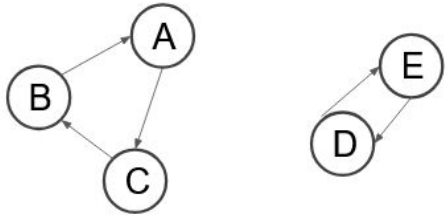


	A	B	C	D	E
A	-	3	2	1	5
B	3	-	2	2	4
C	2	2	-	3	2
D	1	2	3	-	2
E	5	4	2	2	-

¿Cómo evitamos los subtours?

Programación Lineal Entera: Viajante

- *Alternativa 1: Romper los subtours*



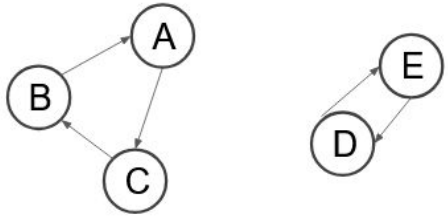
- *Para cada grupo de 2 nodos:*
  - *Yed + Yde <= 1*
- *Para cada grupo de 3 nodos:*
  - *Yab + Yac + Ybc + Yba + Yca + Ycb <= 2*

- *Para cada grupo S:*
  - $\sum Yij \in S <= |S| - 1 \qquad \forall S \subset V,\ 2<=|S|<=|V|/2$

V es el conjunto de vértices, y |V| su dimensión

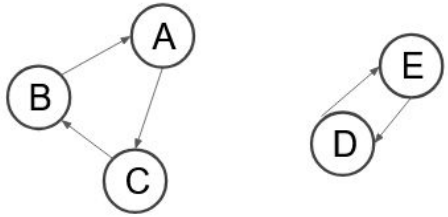
Programación Lineal Entera: Viajante

- *Alternativa 2: Crear los puentes*
- *Forzar la vinculación entre los subgrupos:*
- *Para cada subgrupo S:*
  - $\sum Yij >= 1 \qquad \forall i \in S, j \notin S$



Programación Lineal Entera: Viajante

- *Alternativa 3: Forzar una secuencia*



- *Redefino las variables:*
  - *Yijt: Vale 1 si voy de i a j en el t-ésimo tramo*

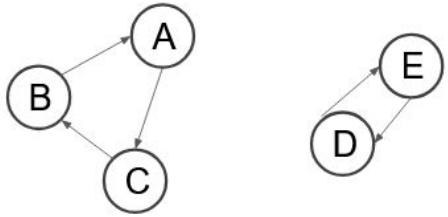
    $\forall\, i,j \in V\,,\ t < |V|$

- *"Si en el paso t llego a j, en el paso t+1 salgo de j":*
  - $\sum Yijt = \sum Yjk(t+1) \qquad \forall\, j \in V\,,\ t < |V|$
  - $\sum YijN = \sum Yjk0 \qquad \forall\, j \in V\,,$

- *Y mantengo la restricción de llegar una vez a cada ciudad*

Programación Lineal Entera: Viajante

- *Alternativa 4: Definir un orden*
- *Agregamos las restricciones MTZ(*):*
  - *Variables U_i: Indican el orden en que visito cada ciudad (menos la inicial)*
  - $U_i - U_j + N\,Y_{ij} <= N\text{-}1$ ($\forall\ i, j$ *salvo el origen)*



(*) Miller, Tucker y Zemlin

Programación Lineal Entera: Viajante

- *Resolución con la biblioteca PULP de Python*

viajante.py

```python
from pulp import *

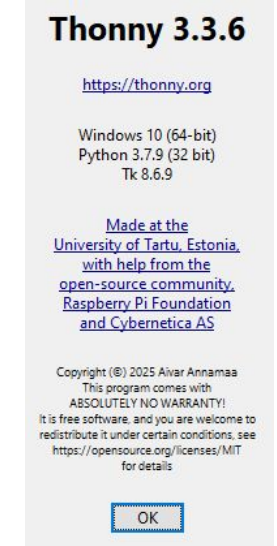
# INICIALIZAR MATRIZ
N=0
D=[]
nombres=[]
arch=open("distancias16.csv","r")
linea=arch.readline()
while linea!='':
    lista=linea.split(";")
    nombres.append(lista.pop(0))
    D.append([])
    for i in lista:
        D[N].append(int(i))
    linea=arch.readline()
    N+=1     # N es la cantidad de ciudades incluyendo el origen
arch.close()

nombres2=[]   #nombres2 es la lista de nombres sin el origen
for n in nombres:
    nombres2.append(n)
nombres2.pop(0)
```

Carga de datos desde un archivo:

- Matriz de distancias
- Lista de nombres

En este código, N es la cantidad de ciudades INCLUYENDO EL ORIGEN



Programación Lineal Entera: Viajante

- *Resolución con la biblioteca PULP de Python*

```
23
24  #CREAR MODELO
25  prob = LpProblem("Viajante",LpMinimize)
26
27  #DEF VARIABLES
28  Y = LpVariable.dicts("Y", (nombres,nombres), 0, cat = "Binary")
29  U = LpVariable.dicts("U", (nombres2), 1, cat = "Integer")
30
31  #FUNCIONAL
32  prob += (lpSum([Y[i][j]*D[nombres.index(i)][nombres.index(j)] for i in nombres for j in nombres]),"Z",)
33
34  #RESTRICCIONES
35  for j in nombres:
36      prob += lpSum([Y[i][j] for i in nombres])==1
37  for i in nombres:
38      prob += lpSum([Y[i][j] for j in nombres])==1
39  for i in nombres:
40      prob += Y[i][i]==0
41  for i in nombres2:
42      for j in nombres2:
43          if i!=j:
44              prob += U[i] - U[j] + (N-1)*Y[i][j] <= N-2
45
```

- Crear un modelo
- Definir variables
- Agregar objetivo
- Agregar restricciones

Programación Lineal Entera: Viajante

- *Resolución con la biblioteca PULP de Python*

```
45
46 prob.writeLP("Viajante.lp")
47
48 #RESOLVER
49 status= prob.solve()
50
51 #MOSTRAR RESULTADO
52 if LpStatus[status] =="Optimal":
53     for i in nombres:
54         for j in nombres:
55             if Y[i][j].value()==1:
56                 print(Y[i][j], D[nombres.index(i)][nombres.index(j)])
57     for i in nombres2:
58         print(U[i], U[i].value())
59     print("Z =", value(prob.objective))
60
```

- Crear un archivo txt con el modelo creado
- Resolver el modelo
- Si hay solución óptima, muestra las variables distintas de cero y las Ui
- Muestra el funcional

Programación Lineal Entera: Viajante

- *Resolución con la biblioteca PULP de Python*

```
Shell
Result - Optimal solution found

Objective value:                9913.00000000
Enumerated nodes:               34493
Total iterations:               936799
Time (CPU seconds):             15.15
Time (Wallclock seconds):       15.15

Option for printingOptions changed from normal to all
Total time (CPU seconds):       15.16   (Wallclock seconds):       15.16

Y_Buenos_Aires_Corrientes 917
Y_La_Plata_Buenos_Aires 58
Y_Santa_Rosa_Neuquén 529
Y_Santa_Fe_Córdoba 369
Y_Paraná_Santa_Fe 31
Y_Córdoba_La_Rioja 450
Y_San_Luis_Santa_Rosa 505
Y_San_Juan_Mendoza 170
Y_Mendoza_San_Luis 258
Y_La_Rioja_San_Juan 444
Y_Viedma_La_Plata 904
Y_Neuquén_Rawson 743
Y_Rawson_Ushuaia 1735
Y_Rio_Gallegos_Viedma 1642
Y_Ushuaia_Rio_Gallegos 579
Y_Corrientes_Paraná 579
U_La_Plata 15.0
U_La_Plata 15.0
U_Santa_Rosa 9.0
U_Santa_Fe 3.0
U_Paraná 2.0
U_Córdoba 4.0
U_San_Luis 8.0
U_San_Juan 6.0
U_Mendoza 7.0
U_La_Rioja 5.0
U_Viedma 14.0
U_Neuquén 10.0
U_Rawson 11.0
U_Rio_Gallegos 13.0
U_Ushuaia 12.0
U_Corrientes 1.0
Z = 9913.0

>>>
```

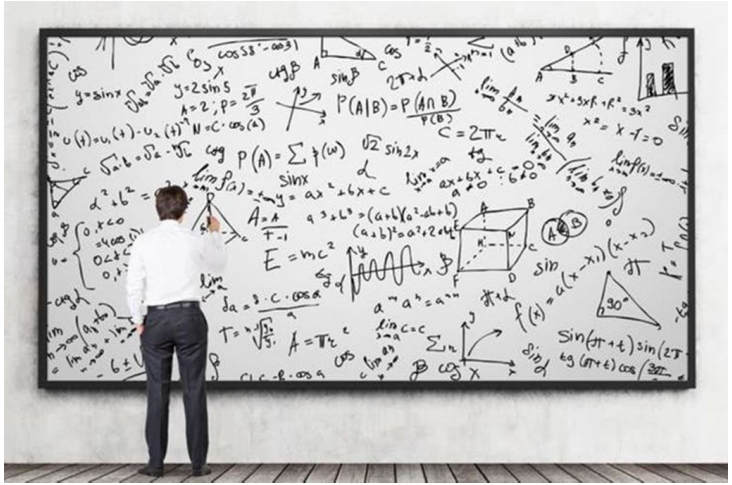
Programación Lineal Entera: Viajante

Tarea para el hogar:

- *Analizar la dimensión de las 4 alternativas propuestas para el problema del viajante:*
  - *Cantidad de variables*
  - *Cantidad de restricciones*

    - *1: Romper los subtours*
    - *2. Crear los puentes*
    - *3. Forzar una secuencia*
    - *4. Definir un orden*

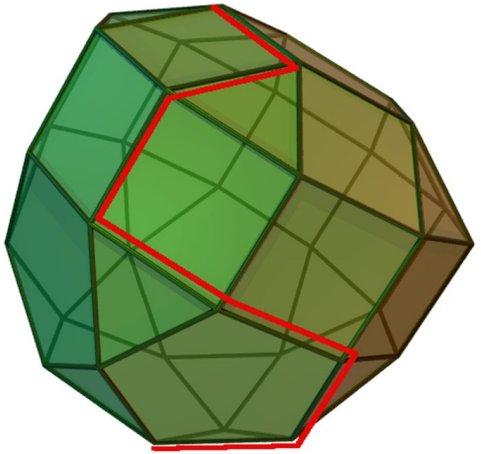
Programación Lineal: Complejidad



Programación Lineal: Complejidad

- Programación Lineal continua:

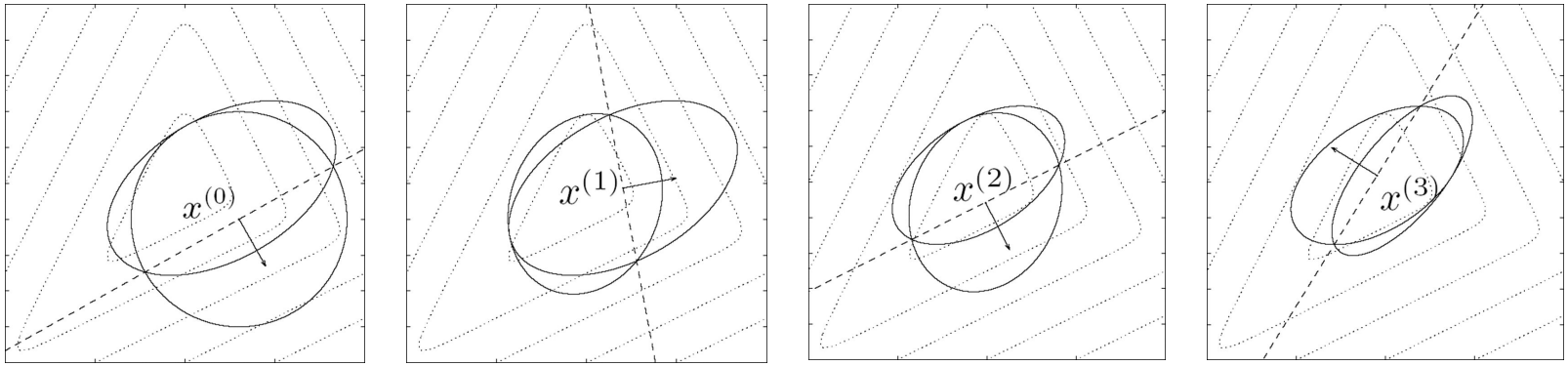
Método simplex (1947): Complejidad exponencial



Programación Lineal: Complejidad

- Programación Lineal continua:

Método del elipsoide (1979): Polinomial

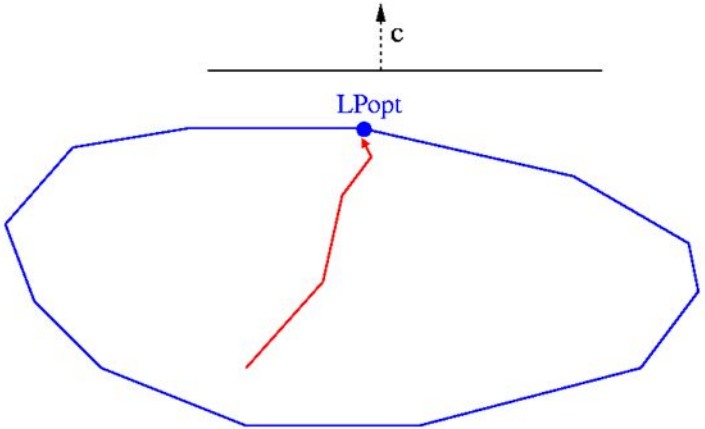


En la práctica no es útil, pero demuestra que la PL es Polinomial

Programación Lineal: Complejidad

- Programación Lineal continua:

Método del punto interior (1984): a partir del elipsoide



Este sí tiene uso práctico, junto con Simplex

Programación Lineal: Complejidad

- ¿Por qué usamos Simplex entonces?

  Porque FUNCIONA

  - Suele resolver problemas complejos en 3m pasos (siendo m la cantidad de restricciones)
  - Da información adicional como costos de oportunidad
  - Permite modificar el planteo de problemas ya resueltos



¿Y el nivel de complejidad?

Programación Lineal: Complejidad

- Programación Lineal Entera

  A cada variable bivalente se le asigna valor 0 y se resuelve. Luego se le asigna 1 y se resuelve otra vez.
  Si tengo una bivalente, resuelvo 2 veces el problema. Si tengo 4 variables, lo resuelvo 16 veces.

  O sea, cada problema entero equivale a 2^n problemas continuos

  O sea, es muy exponencial.



¡Muchas gracias!